



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Sunita Aqila
NPM	2428240072
Kelas	SI5B
Topik Tugas 1	No.16 Kampus – Ruang Kelas
Entitas utama	Ruang Kelas
Tautan repositori	https://github.com/oreo220/si5B_kuis1_SunitaAqila
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	08 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap

Bab	Isi	Bukti utama
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	No. 16 Kampus – Ruang Kelas
Entitas utama (tunggal)	Ruang Kelas
Alamat dasar	/classrooms
Field data	id, kodeRuang, gedung, lantai, kapasitas, adaProyektor
Field wajib saat POST	kodeRuang, gedung, kapasitas
Query string untuk filter	gedung
Data awal	3 data ruang kelas
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[classrooms]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[classrooms]/:id	Tidak	200	Ambil satu data
3	POST	/[classrooms]	Ya	201	Tambah data
4	PUT	/[classrooms]/:id	Ya	200	Ubah data
5	DELETE	/[classrooms]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah node -v, npm -v, npm init -y, dan npm install express cors dotenv beserta keluarannya sampai selesai.

```
C:\Users\User\Downloads>mkdir si5b-kuis1-sunitaaqila
C:\Users\User\Downloads>cd C:\Users\User\Downloads\si5b-kuis1-sunitaaqila
C:\Users\User\Downloads\si5b-kuis1-sunitaaqila>npm init -y
Wrote to C:\Users\User\Downloads\si5b-kuis1-sunitaaqila\package.json:

{
  "name": "si5b-kuis1-sunitaaqila",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

C:\Users\User\Downloads\si5b-kuis1-sunitaaqila>npm install express cors dotenv
added 71 packages, and audited 72 packages in 3s
30 packages are looking for funding
  run 'npm fund' for details
found 0 vulnerabilities

C:\Users\User\Downloads\si5b-kuis1-sunitaaqila>npm install --save-dev nodemon
added 26 packages, and audited 98 packages in 2s
35 packages are looking for funding
  run 'npm fund' for details
3 high severity vulnerabilities
To address all issues (including breaking changes), run:
  npm audit fix --force
Run 'npm audit' for details.
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

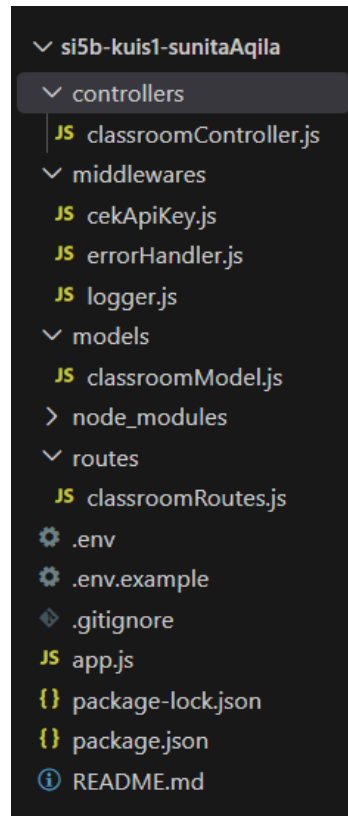
Express digunakan untuk membuat server dan RESTful API, CORS digunakan untuk mengatur akses request dari sumber yang berbeda, sedangkan dotenv digunakan untuk membaca konfigurasi dari file `.env`. Versi Node.js perlu memenuhi syarat Express 5 agar Express dapat berjalan dengan baik dan fitur yang digunakan didukung oleh versi Node.js tersebut.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

- **models:** digunakan untuk mengelola data ruang kelas dan operasi terhadap data tersebut.
- **controllers:** digunakan untuk memproses request dan menjalankan logika dari setiap endpoint.
- **routes:** digunakan untuk menentukan endpoint API dan menghubungkannya dengan controller.
- **middlewares:** digunakan untuk menjalankan fungsi tambahan seperti logger, pengecekan API key, dan penanganan error.

2.2 Ringkasan tugas setiap berkas

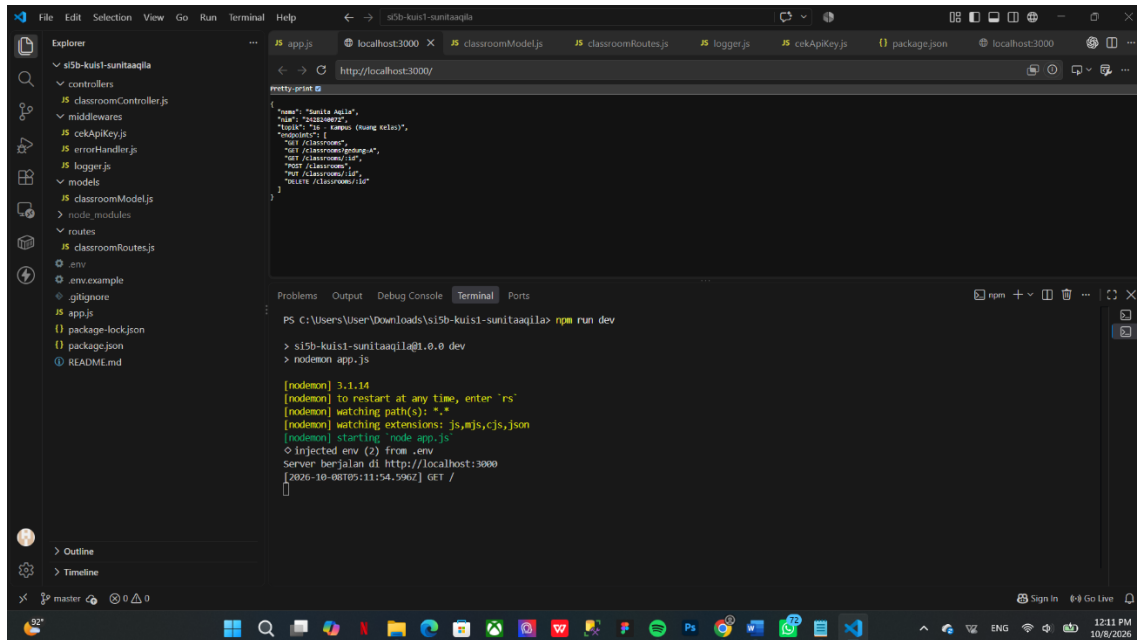
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/[classroom]Model.js	Berisi data classrooms dan fungsi getAll(), getById(), create(), update(), serta remove() untuk mengelola data ruang kelas.
controllers/[classroom]Controller.js	Berisi data classrooms dan fungsi getAll(), getById(), create(), update(), serta remove() untuk mengelola data ruang kelas.
routes/[classroom]Routes.js	Berisi route GET, POST, PUT, dan DELETE untuk resource /classrooms serta menggunakan middleware cekApiKey pada POST, PUT, dan DELETE.
middlewares/logger.js	Berisi fungsi logger() yang mencatat waktu, method, dan URL setiap request yang masuk ke server.
middlewares/cekApiKey.js	Berisi fungsi cekApiKey() yang memeriksa header x-api-key dan membandingkannya dengan process.env.API_KEY.
middlewares/errorHandler.js	Berisi fungsi errorHttp(), notFoundHandler(), dan errorHandler() untuk membuat serta menangani error secara terpusat.
app.js	Digunakan untuk membuat aplikasi Express, memasang middleware, menghubungkan classroomRoutes, dan menjalankan server menggunakan process.env.PORT.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah npm start atau npm run dev dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port berasal dari variabel PORT pada file .env, yaitu 3000, yang dibaca melalui process.env.PORT pada app.js. Jika port tersebut sudah digunakan oleh aplikasi lain, server tidak dapat berjalan pada port yang sama dan akan terjadi konflik port. Solusinya adalah menghentikan aplikasi yang menggunakan port tersebut atau menggunakan nomor port lain.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

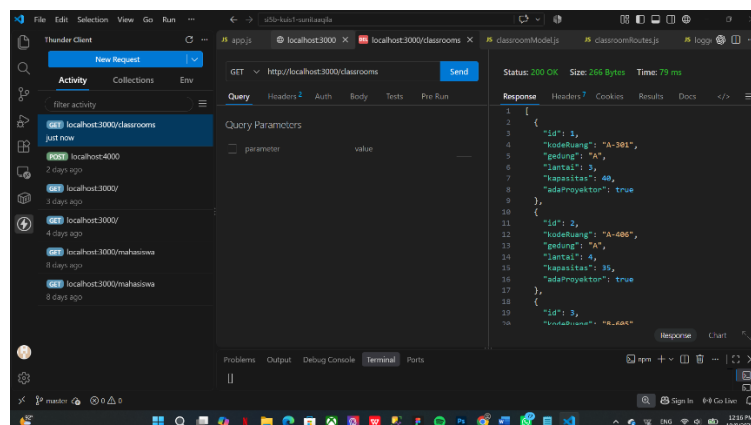
Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji". Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[classrooms])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /[classrooms]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	Status: 200 OK Size: 266 Bytes Time: 79 ms
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

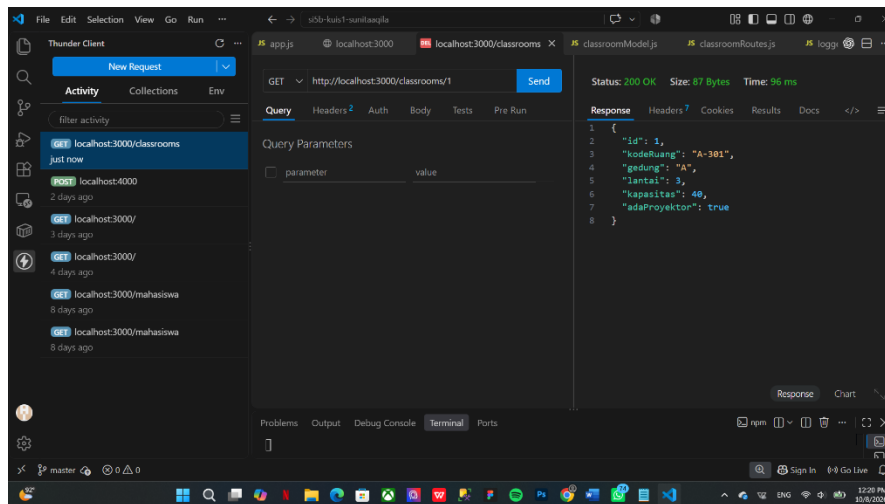
Penjelasan:

Pada pengujian ini saya menggunakan GET /classrooms untuk mengambil seluruh data ruang kelas. Request masuk ke router.get("/"), kemudian diteruskan ke fungsi classroomController.getAll. Controller mengambil nilai req.query.gedung dan memanggil classroomModel.getAll(gedung) untuk mendapatkan data dari array classrooms. Karena tidak menggunakan filter gedung, seluruh data ruang kelas dikembalikan dengan status 200.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /[classrooms]/[1]
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	Status: 200 OK Size: 266 Bytes Time: 79 ms
Lulus?	☒ Ya ☐ Tidak



Gambar 5. Hasil GET berdasarkan ID

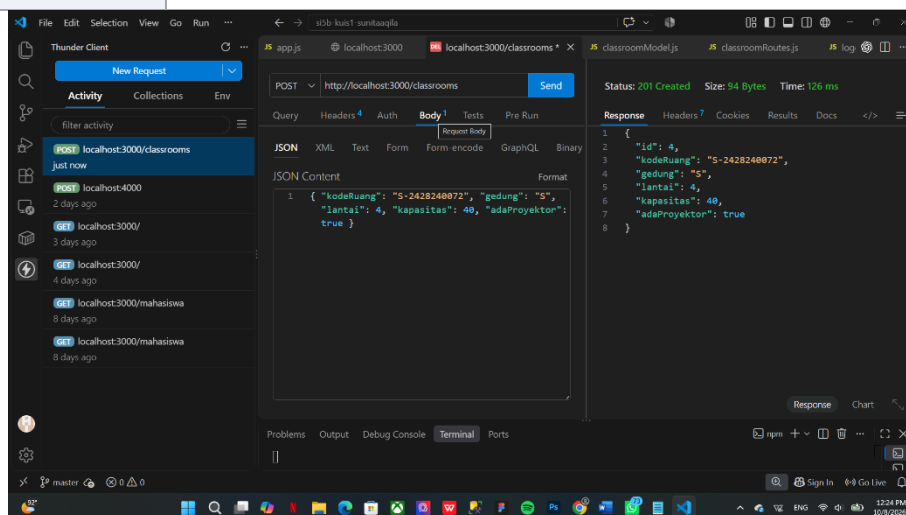
Penjelasan:

Pada pengujian ini saya menggunakan GET /classrooms/1 untuk mengambil satu ruang kelas berdasarkan ID. Nilai 1 diambil dari URL melalui req.params.id, kemudian diubah menjadi angka menggunakan parseInt() karena ID pada data model berupa number. Setelah itu controller memanggil classroomModel.getById(id) untuk mencari data yang sesuai dan mengembalikannya dengan status 200.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /[classrooms]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field wajib 1]": "...", "[field wajib 2]": "..." }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	Status: 201 Created Size: 94 Bytes Time: 126 ms
Lulus?	☒ Ya ☐ Tidak



Gambar 6. Hasil POST data baru

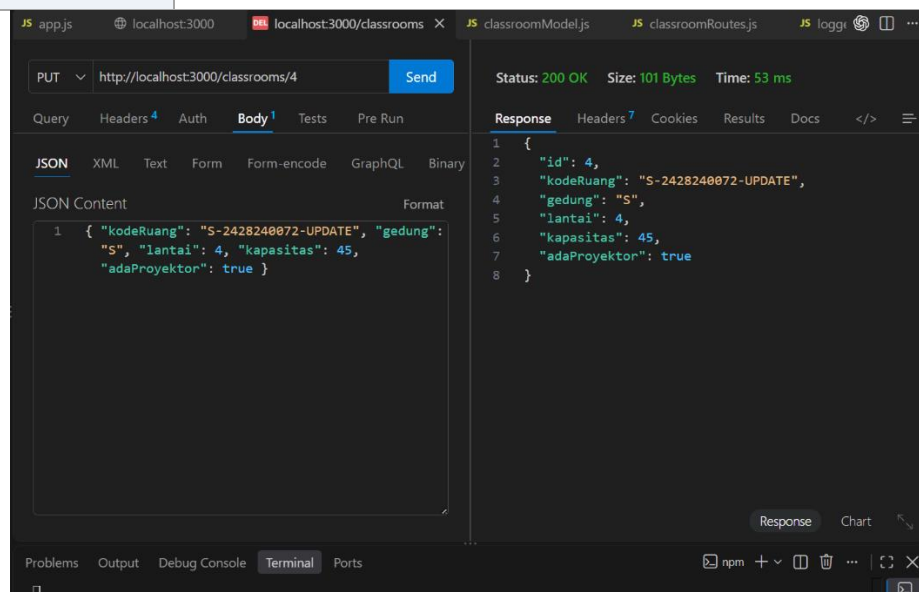
Penjelasan:

Pada pengujian POST, saya mengirim header x-api-key dengan nilai yang sesuai dengan API_KEY pada file .env. Header tersebut diperiksa oleh middleware cekApiKey sebelum request diteruskan ke controller. Data JSON dapat dibaca oleh Express karena menggunakan express.json(). Setelah validasi berhasil, controller memanggil classroomModel.create() untuk menambahkan data dan server memberikan status 201 Created karena terdapat resource ruang kelas baru yang berhasil dibuat. Data kodeRuang menggunakan NIM saya sebagai identitas pengujian.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /[classrooms]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "[field]": "[nilai baru]" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	Status: 200 OK Size: 101 Bytes Time: 53 ms
Lulus?	☒ Ya ☐ Tidak



Gambar 7. Hasil PUT mengubah data

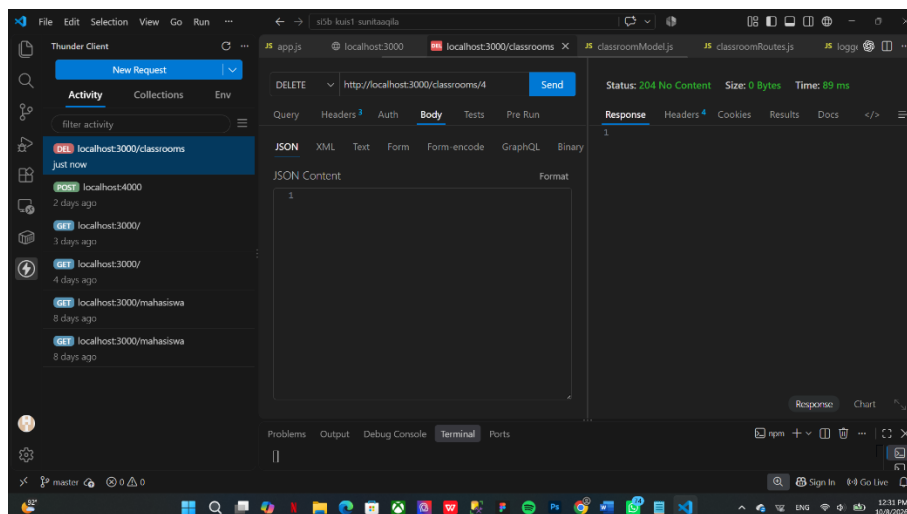
Penjelasan:

Pada pengujian PUT, saya menggunakan ID data yang sebelumnya dibuat pada T03. Field kodeRuang dan kapasitas diubah sesuai data pada request, sedangkan field gedung, lantai, dan adaProyektor tetap sesuai data yang dikirim. Request terlebih dahulu diperiksa oleh middleware cekApiKey, kemudian controller melakukan validasi dan memanggil classroomModel.update() untuk mengubah data. Setelah perubahan berhasil, server mengembalikan objek yang sudah diperbarui dengan status 200.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /[classrooms]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	☒ Ya ☐ Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Pada pengujian ini saya menggunakan method DELETE dengan ID data yang dibuat pada T03 dan mengirimkan x-api-key yang valid. Request diperiksa oleh middleware cekApiKey, kemudian controller mencari data dan memanggil `classroomModel.remove()` untuk menghapusnya. Setelah berhasil dihapus, server memberikan status 204 No Content. Status 204 menunjukkan bahwa request berhasil diproses tetapi server tidak mengembalikan isi response, sehingga response body dibiarkan kosong.

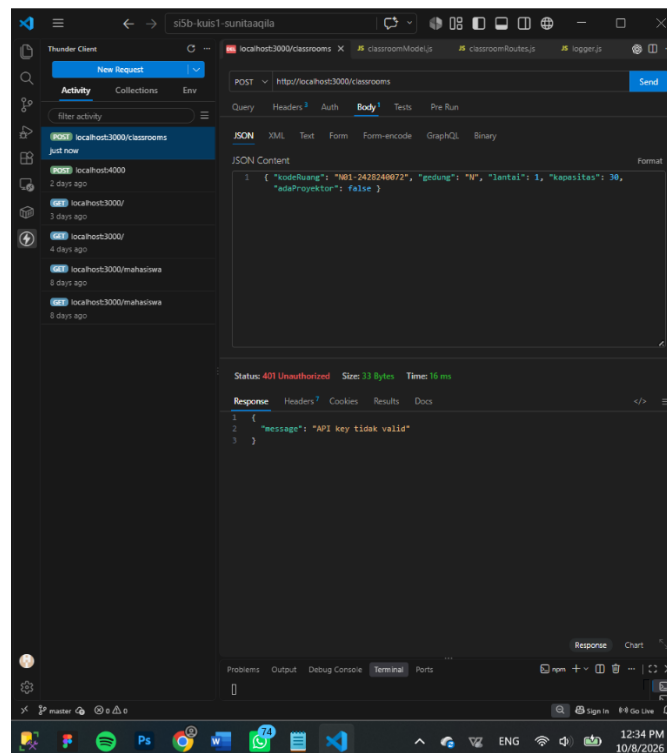
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (<i>request</i>)	POST /[classrooms] (tanpa x-api-key)
Header	Content-Type: application/json
Body	{ data yang valid }
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized — "API key tidak valid"
Lulus?	☒ Ya ☐ Tidak



Gambar 9. POST tanpa API key ditolak

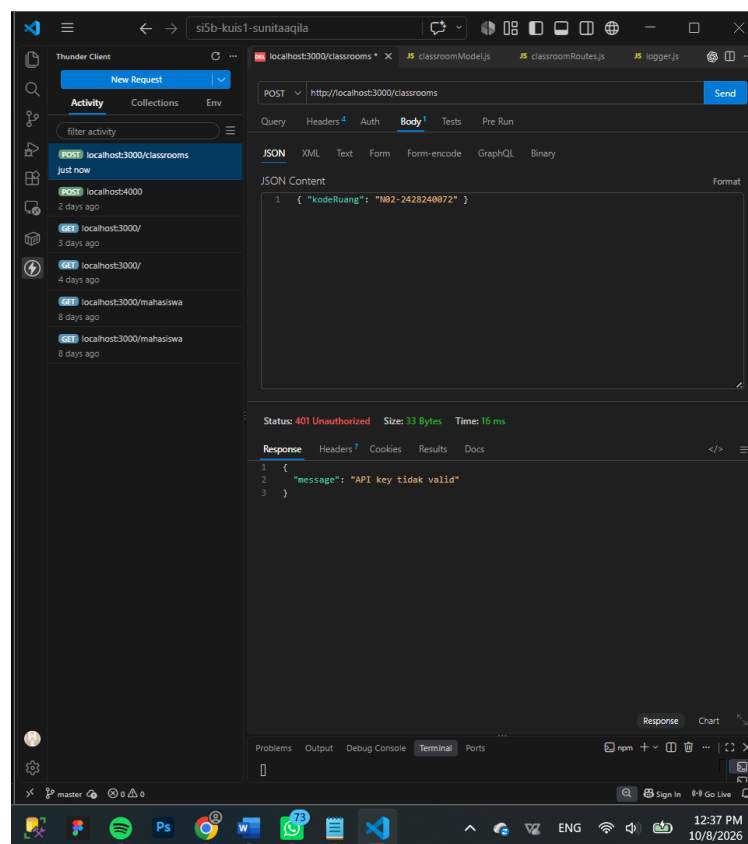
Penjelasan:

Pada pengujian ini request POST tidak menyertakan header x-api-key. Request ditolak oleh middleware cekApiKey karena nilai API key tidak tersedia atau tidak sesuai dengan process.env.API_KEY. Karena middleware tidak menjalankan next(), request tidak diteruskan ke classroomController.create, sehingga data tidak ditambahkan. Server memberikan status 401 Unauthorized.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /[classrooms]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ hanya 15edung15n field wajib }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request — “kodeRuang, 15edung, dan kapasitas wajib diisi”
Lulus?	☒ Ya ☐ Tidak



Gambar 10. POST dengan body tidak lengkap

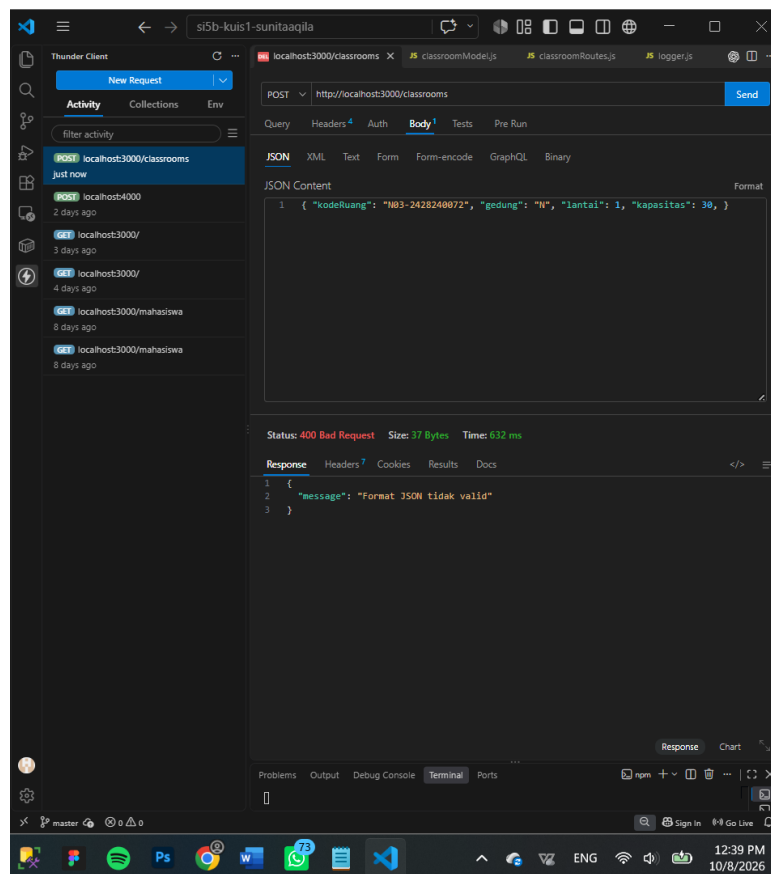
Penjelasan:

Pada pengujian ini API key yang digunakan sudah benar sehingga request dapat diteruskan ke controller. Namun body hanya berisi kodeRuang dan tidak memiliki field wajib gedung serta kapasitas. Validasi dilakukan pada layer **Controller**, yaitu `classroomController.create`. Karena data tidak memenuhi ketentuan, controller meneruskan error dengan status 400 Bad Request dan data tidak ditambahkan.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /[classrooms]
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ JSON dengan koma berlebih }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request — "Format JSON tidak valid"
Lulus?	☒ Ya ☐ Tidak



Gambar 11. POST dengan JSON rusak

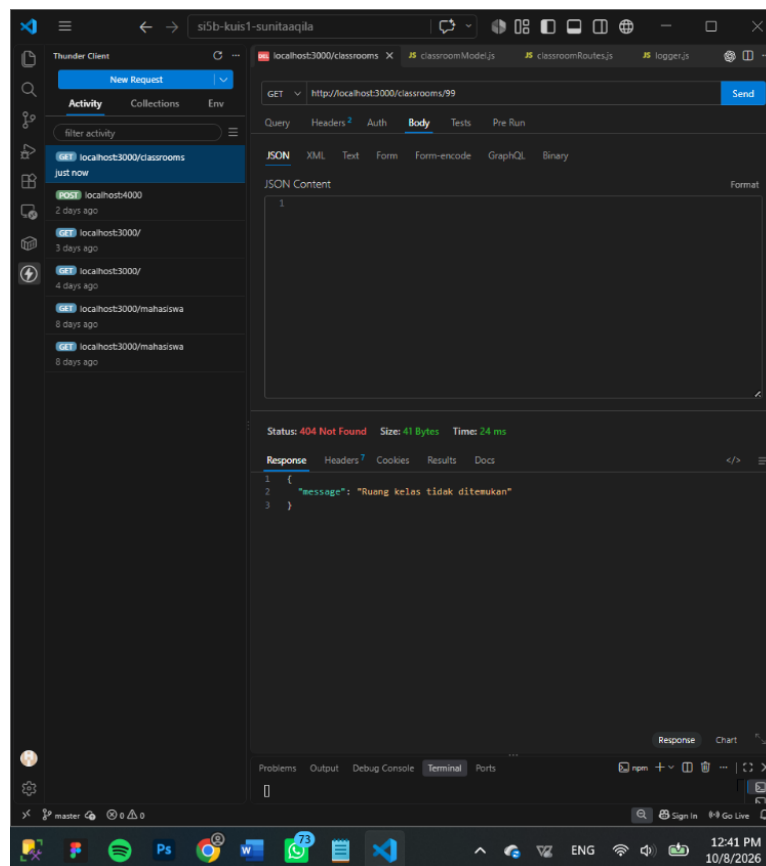
Penjelasan:

Pada pengujian ini JSON yang dikirim sengaja dibuat tidak valid karena terdapat koma berlebih. JSON tersebut dideteksi oleh middleware `express.json()` sebelum request mencapai Controller. Error kemudian diteruskan ke `errorHandler`. Middleware `errorHandler` mengenali error dengan tipe `entity.parse.failed` dan mengubahnya menjadi response JSON dengan status 400 serta pesan "Format JSON tidak valid". Dengan demikian, server tidak mengembalikan halaman HTML atau stack trace.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /[classrooms]/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found — "Ruang kelas tidak ditemukan"
Lulus?	☒ Ya ☐ Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Pada pengujian ini saya menggunakan ID 99 yang tidak terdapat pada data ruang kelas. Route meneruskan request ke `classroomController.getById`. ID dari `req.params.id` diubah menjadi angka menggunakan `parseInt()`, kemudian dicari melalui `classroomModel.getById()`. Karena data tidak ditemukan, Controller membuat error menggunakan `errorHttp(404, "Ruang kelas tidak ditemukan")` dan meneruskannya ke `errorHandler`. Selanjutnya `errorHandler` mengirim response JSON dengan status 404 Not Found.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Ya
T02	Menampilkan satu data berdasarkan ID	200	200	Ya
T03	Menambah data baru	201	201	Ya
T04	Mengubah data	200	200	Ya
T05	Menghapus data	204	204	Ya
N01	Menambah data tanpa API key	401	401	Ya
N02	Menambah data dengan body tidak lengkap	400	400	Ya
N03	Mengirim JSON yang rusak	400	400	Ya
N04	Mengambil ID yang tidak ada	404	404	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/oreo220/si5B_kuis1_SunitaAqila
Pesan commit terakhir	menyesuaikan response DELETE menjadi 204
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Pada API ruang kelas saya, Route bertugas menentukan endpoint dan mengarahkan request ke Controller, contohnya `router.get("/", classroomController.getAll)`. Controller memproses request dan memanggil Model, contohnya `classroomController.getAll`, sedangkan Model bertugas mengelola data ruang kelas, contohnya fungsi `getAll()` pada `classroomModel.js`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan error lebih rapi karena response error dapat diatur pada satu tempat tanpa menulis kode yang sama di setiap route. Pada pengujian N04, ketika ID 99 tidak ditemukan, Controller meneruskan error ke `errorHandler` dan API menghasilkan response JSON dengan status 404 Not Found.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan topik Tugas 1 saya	☒
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	☒
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	☒
4	Identitas saya (nama atau NIM) muncul pada data uji POST	☒
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	☒
6	Tabel rekapitulasi sesuai dengan tangkapan layar	☒
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	☒
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	☒

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggah	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100